Week 1 – Class 1

Mental Models & LLM Mechanics

From AI user → AI architect mindset

Today you will learn to:

- Understand what an LLM actually is
- Explain tokens, temperature, and token cost
- Understand why hallucination is structural
- Separate AI model from AI system thinking

First principle: never say “AI is smart.” Say “AI predicts statistically likely continuations.”

What is a Large Language Model?

LLM = probabilistic next-token prediction engine

- Trained on massive text corpora using transformer architecture
- Does not “know” facts the way humans do
- Predicts the most statistically likely next token
- Optimizes for likelihood of continuation, not truth

Teaching line: “LLMs optimize for likelihood, not truth.”

Next-token prediction in simple terms

A model chooses among probabilities over vocabulary

- Prompt: “The capital of France is …”
- Model may assign Paris → high probability, London → lower, Berlin → lower
- At low temperature, it picks the highest-probability path more often
- There is no hidden “knowledge database” inside the prompt call

```
import random

distribution = {"Paris": 0.91, "London": 0.03, "Berlin": 0.02}
print(random.choices(list(distribution.keys()),
                     weights=list(distribution.values()))[0])
```

Micro demo: probability ≠ knowledge

Tokens: what the model actually sees

Models process tokens, not words

- A token is a chunk of text, often a subword unit
- “Artificial Intelligence” may become multiple tokens
- Words, punctuation, and spacing can all affect tokenization
- Tokens drive cost, latency, and context window usage

Production mindset: token management = cost management + performance management + reliability management.

Transformer (conceptual only)

Attention tells the model which previous tokens matter most right now

- Transformer = attention mechanism + parallel processing
- Example: “I went to the bank to deposit money.”
- “Deposit money” pushes attention toward financial bank, not river bank
- This is why context matters so much in modern LLM systems

Teaching line: “Context is king.”

Context window

Every model has a maximum token limit

- Examples: 8K, 32K, 128K token windows depending on model family
- If you exceed the limit, older context may be truncated or the request may fail
- Long prompts increase cost, memory pressure, and latency
- This matters heavily in production system design

```
short_prompt = "Explain AI"
long_prompt = "Explain artificial intelligence in simple terms with 3 examples and a short summary for software engineers."

print(len(short_prompt))
print(len(long_prompt))
```

Micro demo: short prompt vs long prompt

Hallucination: why it happens

Hallucination is not a bug. It is a design consequence.

- The model predicts plausible continuation, not verified truth
- It does not automatically verify facts
- It does not automatically access real-time or private knowledge
- Likelihood without verification creates plausible but unsupported answers

Teaching line: “Hallucination is probability without verification.”

AI model vs AI system

This distinction is the core of the course

AI model

- Predictive engine
- Stateless
- Probabilistic
- Generates tokens

AI system

- Model + retrieval
- Guardrails + API
- Logging + UI
- Cost management + workflows

Engineers do not deploy models. Engineers deploy systems.

Live API demo pattern

Validate environment first. Then connect the client.

```
import os
from openai import OpenAI

api_key = os.getenv("OPENAI_API_KEY")
if not api_key:
    raise EnvironmentError("OPENAI_API_KEY not found")

client = OpenAI(api_key=api_key)
```

First real API call

Show response content and token usage.

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a precise AI systems instructor."},
        {"role": "user", "content": "Explain hallucination in one sentence."}
    ],
    temperature=0.0
)

print(response.choices[0].message.content)
print(response.usage)
```

Temperature = variability control

Temperature controls randomness, not truth

- 0.0 → more deterministic, more repeatable
- 0.8 → more variation, more creative phrasing
- Higher temperature can increase plausible-but-wrong outputs indirectly
- Prompt tokens usually stay the same; completion tokens often change

```
for temp in [0.0, 0.8]:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "Explain hallucination in one sentence."}],
        temperature=temp
    )
    print(temp, response.choices[0].message.content)
    print(response.usage)
```

Why token usage matters in production

Tokens affect more than cost

- More tokens = higher cost
- More tokens = more latency
- Long prompts consume limited context budget
- Unbounded token usage creates unstable production systems

Prompt tokens = input cost. Completion tokens = output cost. Total tokens = both together.

Architecture sketch

A simple mental model for modern AI systems

- Basic: User → Backend → LLM → Response
- Expanded: User → Backend → Retrieval → LLM → Guardrails → Response
- Architecture is what reduces hallucination and improves reliability
- This is why systems thinking matters more than “prompt tricks”

Teaching line: “AI systems are probability plus architecture.”

AI is probability.

Deterministic does not mean correct. It means consistently predictable.